

L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765



MECHTRON 3TB4
Lab Tutorial 1 Report
Lab 1 Prelab Report

Date: January 29, 2026

Lab Tutorial 1 Report

Overview

In this tutorial, Quartus Prime was used to design, compile, and verify digital logic circuits using Verilog HDL (Hardware Description Language). A Quartus project was created, Verilog modules were written and compiled for the Cyclone V FPGA (Field-Programmable Gate Array) on the DE1-SoC board, and functionality was verified using waveform-based functional simulation. Additionally, the tutorial introduced pin assignments and the analysis of compilation reports.

Light Switch Circuit

The Light Switch Circuit uses combinational digital logic to implement an exclusive OR (XOR) function, where the output LED is on only when one input is high.

Table 1. Truth table for the Light Switch Circuit.

x1	x2	f
0	0	0
0	1	1
1	0	1
1	1	0

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Jan 21 12:20:23 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	light
Top-level Entity Name	light
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	1 / 32,070 (< 1 %)
Total registers	0
Total pins	3 / 457 (< 1 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1. Screenshot of the Light Switch Circuit compilation report.

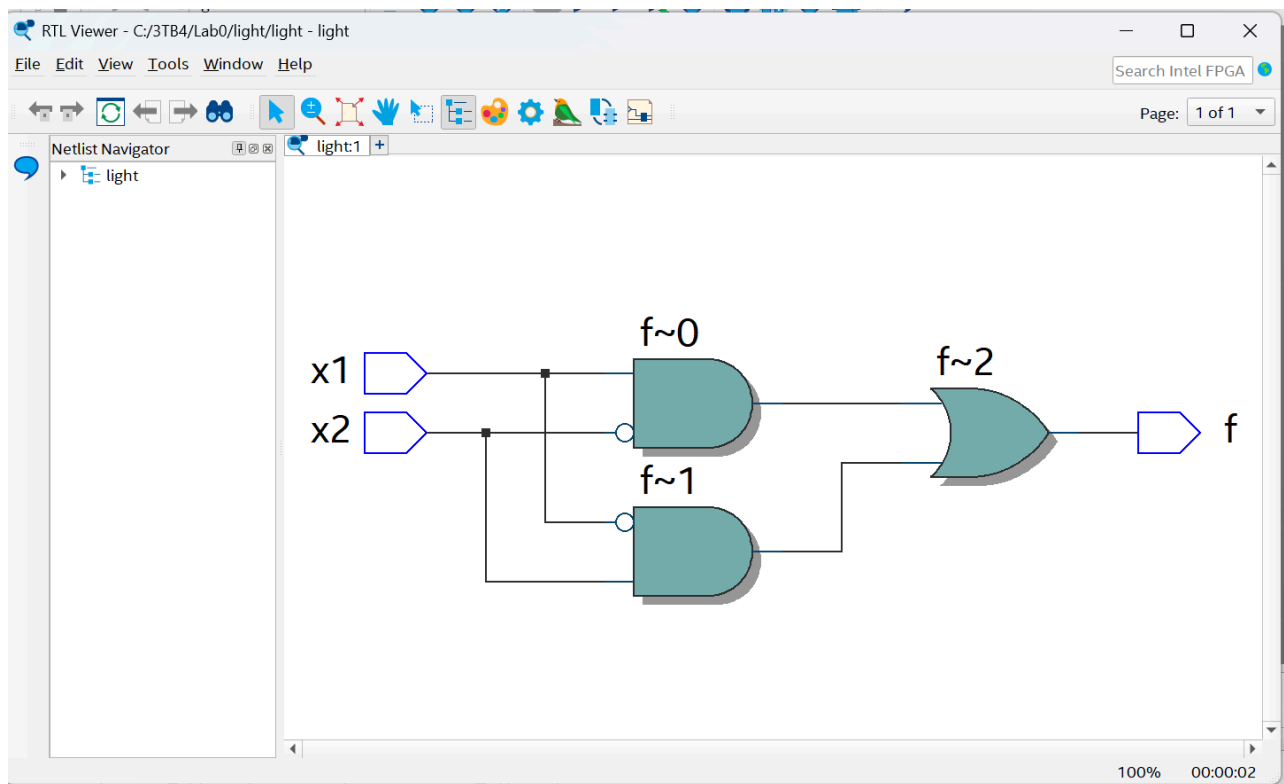


Figure 2. Screenshot of the implemented Light Switch Circuit.

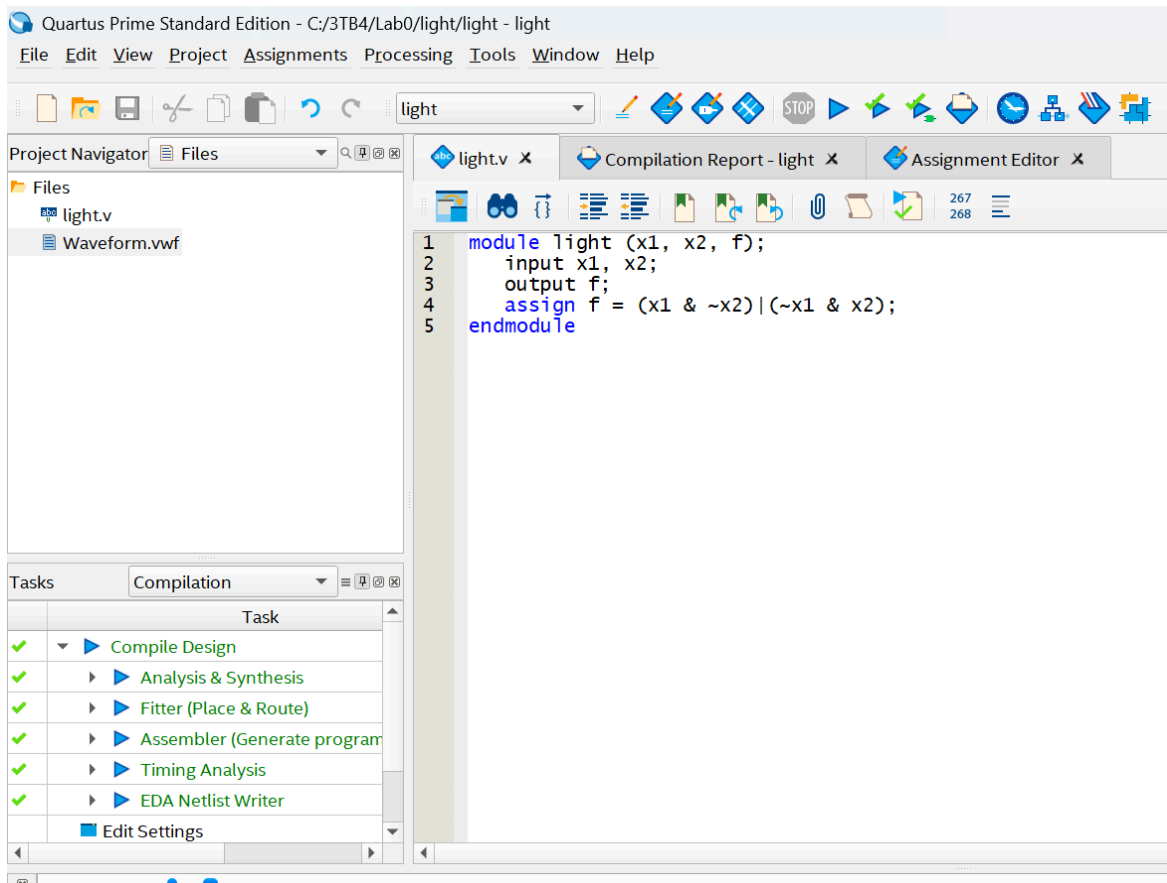


Figure 3a. Screenshot of the module of the Light Switch Circuit.

light.v x Compilation Report - light x Assignment Editor x								
<<new>> Filter on node names: *								
.tatu	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1 ✓		out f	Location	PIN_V16	Yes			
2 ✓		in x1	Location	PIN_AB12	Yes			
3 ✓		in x2	Location	PIN_AC12	Yes			
4	<<new>>	<<new>>	<<new>>					

Figure 3b. Screenshot of the pins used for the Light Switch Circuit.

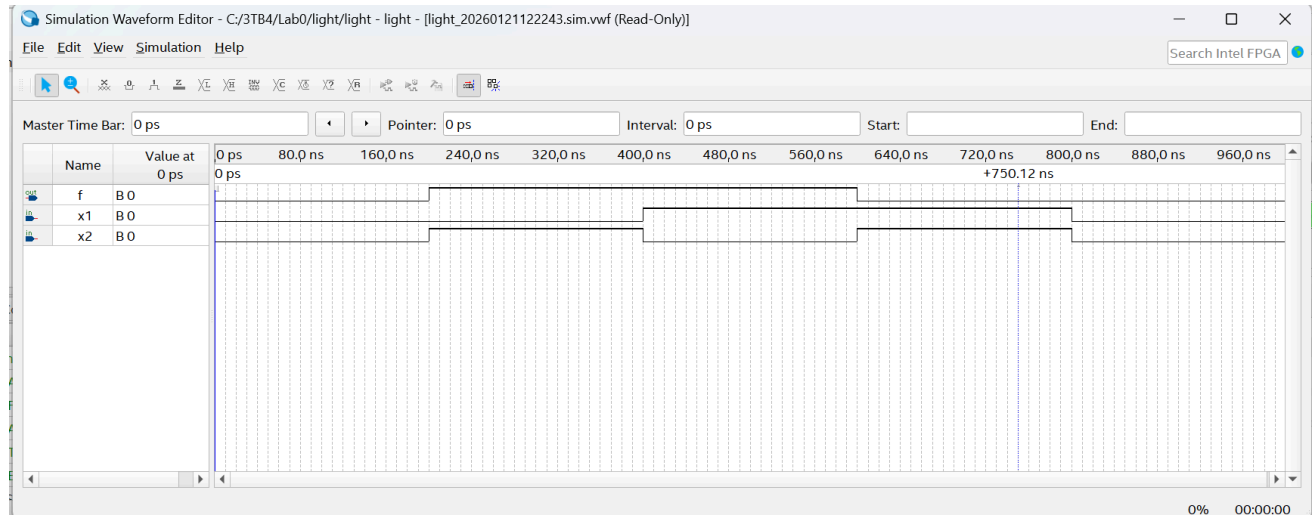


Figure 4. Screenshot of the simulation of the Light Switch Circuit.

Majority Circuit

The Majority Circuit uses combinational digital logic and produces an output of high when two or more of the three inputs (x1, x2, x3) are high. The simulation waveform editor tool was used to verify the circuit design, and the features of the tool were explored using the waveform of the Majority Circuit.

Table 2. Truth table for the Majority Circuit.

x1	x2	x3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

majority3.v

Compilation Report - majority3

Table of Contents

Flow Summary

Flow Settings

Flow Non-Default Global Settings

Flow Elapsed Time

Flow OS Summary

Flow Log

Analysis & Synthesis

Fitter

Assembler

Timing Analyzer

EDA Netlist Writer

Flow Messages

Flow Suppressed Messages

Flow Summary

<<Filter>>

Flow Status

Successful - Mon Jan 26 22:59:13 2026

Quartus Prime Version

23.1std.0 Build 991 11/28/2023 SC Standard Edition

Revision Name

majority3

Top-level Entity Name

majority3

Family

Cyclone V

Device

5CSEMA5F31C6

Timing Models

Final

Logic utilization (in ALMs)

1 / 32,070 (< 1 %)

Total registers

0

Total pins

4 / 457 (< 1 %)

Total virtual pins

0

Total block memory bits

0 / 4,065,280 (0 %)

Total DSP Blocks

0 / 87 (0 %)

Total HSSI RX PCSs

0

Total HSSI PMA RX Deserializers

0

Total HSSI TX PCSs

0

Total HSSI PMA TX Serializers

0

Total PLLs

0 / 6 (0 %)

Total DLLs

0 / 4 (0 %)

Figure 5. Screenshot of the Majority Circuit compilation report.

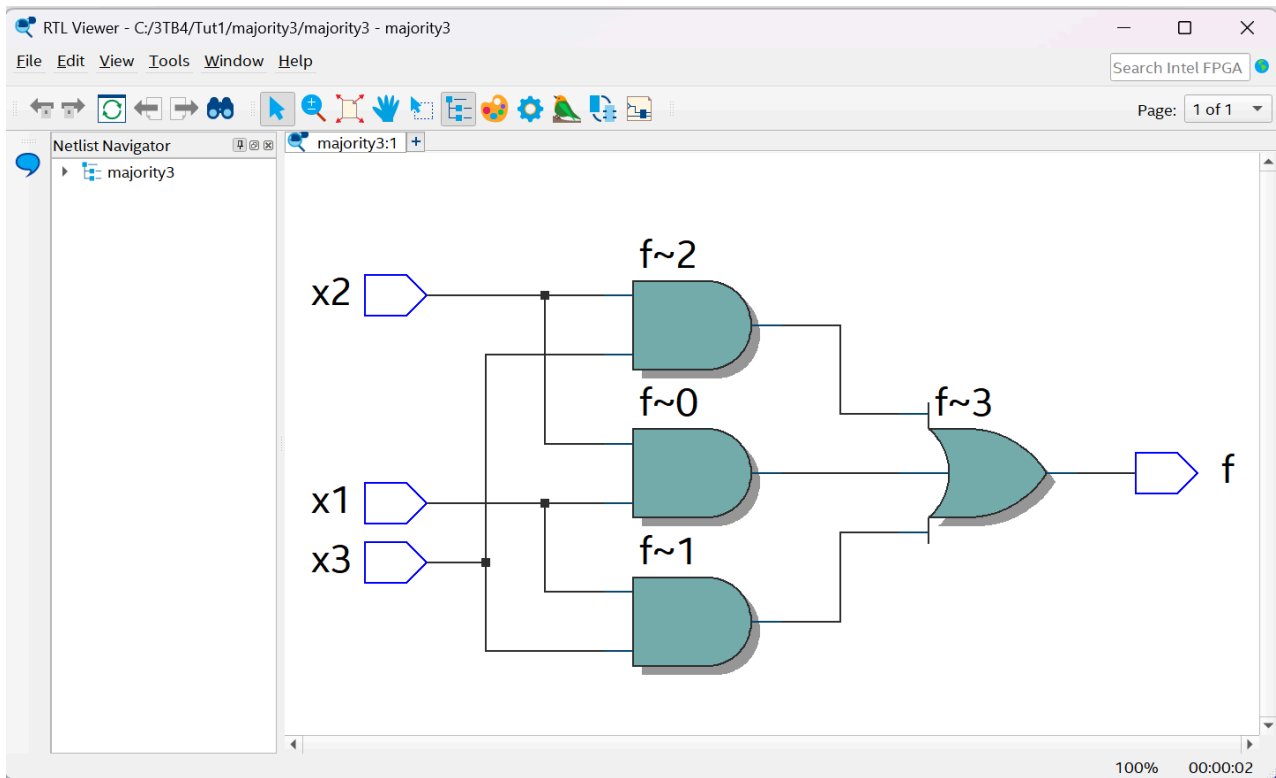


Figure 6. Screenshot of the implemented Majority Circuit.

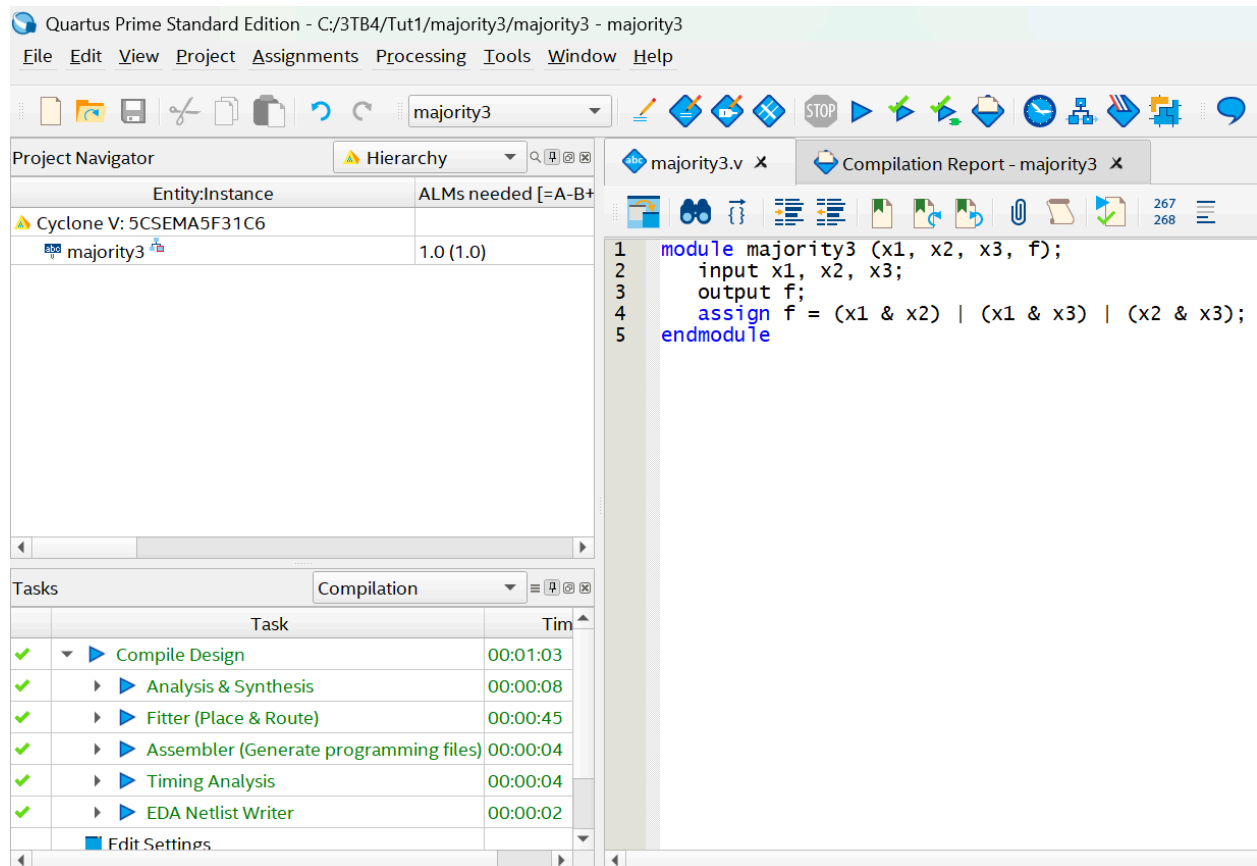


Figure 7. Screenshot of the module of the Majority Circuit.

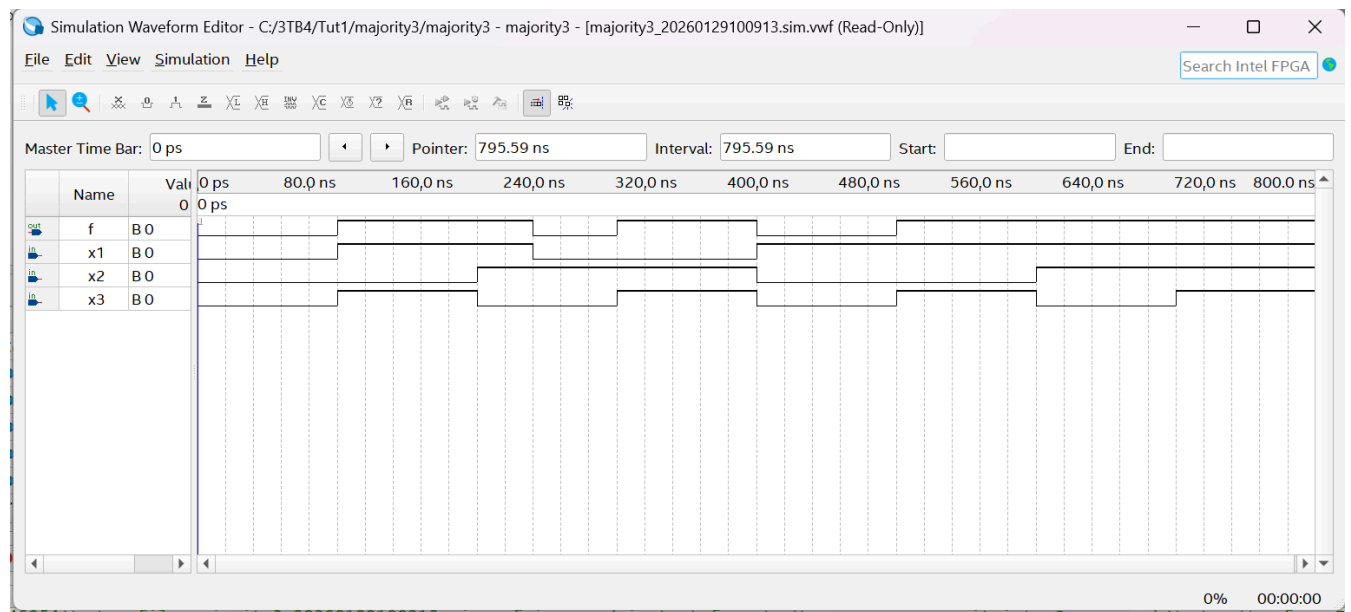


Figure 8. Screenshot of the simulation of the Majority Circuit.

Lab 1 Prelab Report

Part 3: Pre-lab Questions

1. What is the reg data type and what is the wire data type in Verilog?

In Verilog, the reg data type stands for register, and is used for variables that can store value(s) [1]. For example, they are used for latches, flip-flops, and procedural variables. The wire data type is used to connect physical hardware components, making them analogous to physical wires in circuits.

2. Can the wire data type be used on the left side of the assignment statement in a procedural block?

Variables of the data type “wire” cannot be assigned in a procedural block, and they must be assigned outside, as it represents a physical connection [2].

However, variables of the data type “reg” can be assigned in a procedural block, as they can be used to store values.

3. What are the rules for module port connection?

Module port connections must follow port directions and data types. Input ports are type “net” internally, or driven externally by “net” or “reg” types. Output ports may be declared as “net” or “reg” types internally, but must be connected externally to type “net”. Inout ports must always be type “net”. Port widths should be the same size, but connecting different widths is allowed, but may result in synthesis warnings or errors. Unconnected ports are allowed by using “,” and may be left open during instantiation. The “net” data types are required for signals driven by structural connections [3].

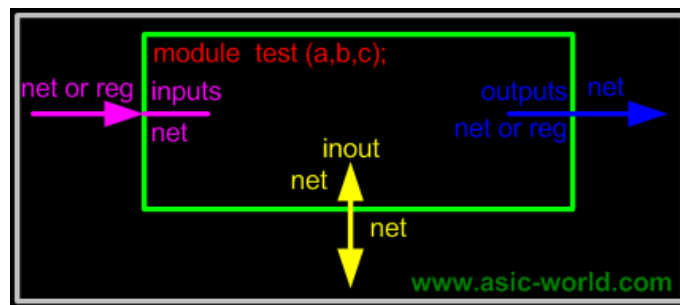


Figure 9. An image to describe the rules for module port connection.

4. What are continuous assignments, blocking assignment and nonblocking assignments?

Continuous assignments are denoted by “assign” and drive values into a wire. Blocking assignments are denoted using “=”. They execute sequentially in procedural blocks of code. Nonblocking assignments are denoted using “<=” and they are used for sequential logic where updates to variables must occur simultaneously [4].

5. What is the difference in procedural coding when implementing combinational logic and sequential logic?

For combinational logic, “always @()” blocks are used to specify the input signals that are required to trigger a change in the output. The outputs are assigned based on the current inputs, and blocking assignments are used [5]. For sequential logic, “always” blocks are also used, but they are triggered by the clock edge (posedge or negedge) and usually use nonblocking assignments [5].

6. How does one avoid inferred latches when using Verilog to describe circuits?

To avoid inferred latches when using Verilog to describe circuits, all outputs and conditions must be assigned values in every possible execution path for all “if” and “case” statements, so there is no undefined behaviour. For “case” statements, it is recommended to have a default case for undefined behaviours [6].

7. What is the difference between the operators “<<” and “<<<”?

The “<<” operator is used for logical left shift, and the vacated LSB (Least Significant Bit) positions are filled with zeros. The “<<<” operator is used for arithmetic left shifts, and checks for the signed bits, but unlike the right arithmetic shift operator “>>>”, it does not preserve or extend the sign bit and shifts the bits left and fills the LSB with zeros. Therefore, there's no difference except that the “<<<” does check for signed bits, but has the same result [7].

8. How to declare an array of 6 elements of a 7-bit wire?

To declare an array of 6 elements of a 7-bit wire: “wire [6:0] array [5:0]” [8].

Part 4: Verilog Modules

1. One-bit data width D flip-flop

```
module d_flipflop(input clk, input d, output reg q);
    always@(posedge clk) begin
        q <= d;
    end
endmodule
```

2. One-bit data width D flip-flop with active low synchronous reset

```
module d_ff_sync_reset_low(input clk, input d, input reset, output reg q);
    always@(posedge clk) begin
        if (!reset)
            q <= 1'b0;
        else
            q <= d;
    end
endmodule
```

3. One-bit data width D flip-flop with active low synchronous reset and active low enable

```
module d_ff_sync_reset_enable(input clk, input d, input reset, input enable, output reg q);
    always@(posedge clk) begin
        if (!reset)
            q <= 1'b0;
        else if (!enable)
            q <= d;
    end
endmodule
```

4. D latch with synchronous enable control

```
module d_latch_sync_en(input clk, input d, input enable, output reg q);
    always@(*) begin
        if (clk && enable)
```

```

        q <= d;
    end
endmodule

```

5. 4-to-1 multiplexer

```

module mux4_1(input [1:0] select, input [3:0] in, output reg out);
    always@(*) begin
        case (select)
            2'b00: out = in[0];
            2'b01: out = in[1];
            2'b10: out = in[2];
            2'b11: out = in[3];
            default: out = 1'b0;
        endcase
    end
endmodule

```

6. 4-bit counter with reset and enable controls

```

module counter(input clk, input reset, input enable, output reg [3:0] out);
    always@(posedge clk) begin
        if (reset) begin
            out <= 4'b0000;
        end
        else if (enable) begin
            out <= out + 1'b1;
        end
    end
endmodule

```

Part 5: Truth Table for Seven-Segment Decoder

Table 3. Truth table for the Seven-Segment Decoder for MELOD.

Input (4-bits)	Hex Input (4-bit)	Output Display	Output (7-bits)
0000	0	0	1000000
0001	1	1	1111001
0010	2	2	0100100
0011	3	3	0110000
0100	4	4	0011001
0101	5	5	0010010
0110	6	6	0000010
0111	7	7	1111000
1000	8	8	0000000
1001	9	9	0010000
1010	A	M	1001000
1011	B	E	0000110
1100	C	L	1000111
1101	D	O	1000000
1110	E	D	0100001
1111	F	OFF	1111111

Table 4. Truth table for the Seven-Segment Decoder CYNTH.

Input (4-bits)	Hex Input (4-bit)	Output Display	Output (7-bits)
0000	0	0	1000000
0001	1	1	1111001
0010	2	2	0100100
0011	3	3	0110000
0100	4	4	0011001
0101	5	5	0010010
0110	6	6	0000010
0111	7	7	1111000
1000	8	8	0000000
1001	9	9	0010000
1010	A	C	1000110
1011	B	Y	0010001
1100	C	N	0101011
1101	D	T	0000111
1110	E	H	0001001
1111	F	OFF	1111111

Part 6: Logic Expressions for Truth Table

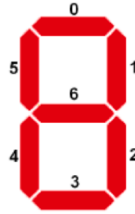


Figure 9. Seven-segment display with the segments labelled.

For the truth tables in the seven-segment decoder, logic 0 indicates the segment is turned on. Segments 0 and 1 are tested using minterms since they can accurately identify all input combinations that cause the segment to be inactive. This offers a correct Boolean expression which can be simplified further.

Due to the limited number of segments, some letters may look different from what is expected. For example, for MELOD, the seven-segment decoder is unable to display “M”, so a lowercase “n” was displayed instead. The letters “E”, “L”, and “O” were displayed in their uppercase form, while “d” was displayed in its lowercase form.

Table 5. Truth table for the Seven-Segment Decoder for segment 0 for MELOD.

X1	X2	X3	X4	Output Display	Segment 0	Minterm
0	0	0	0	0	0	
0	0	0	1	1	1	$\bar{X}_1 \bar{X}_2 \bar{X}_3 X_4$
0	0	1	0	2	0	
0	0	1	1	3	0	
0	1	0	0	4	1	$\bar{X}_1 X_2 \bar{X}_3 \bar{X}_4$
0	1	0	1	5	0	
0	1	1	0	6	0	
0	1	1	1	7	0	
1	0	0	0	8	0	
1	0	0	1	9	0	
1	0	1	0	M	0	
1	0	1	1	E	0	
1	1	0	0	L	1	$X_1 X_2 \bar{X}_3 \bar{X}_4$
1	1	0	1	O	0	
1	1	1	0	D	1	$X_1 X_2 X_3 \bar{X}_4$
1	1	1	1	OFF	1	$X_1 X_2 X_3 X_4$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 X_3 \bar{X}_4 + X_1 X_2 X_3 X_4$$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + X_2 \bar{X}_3 \bar{X}_4 (\bar{X}_1 + X_1) + X_1 X_2 X_3 (\bar{X}_4 + X_4)$$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 X_3$$

Table 6. Truth table for the Seven-Segment Decoder for segment 1 for MELOD.

X1	X2	X3	X4	Output Display	Segment 1	Minterm
0	0	0	0	0	0	
0	0	0	1	1	0	
0	0	1	0	2	0	
0	0	1	1	3	0	
0	1	0	0	4	0	
0	1	0	1	5	1	$\bar{X}_1 X_2 \bar{X}_3 X_4$
0	1	1	0	6	1	$\bar{X}_1 X_2 X_3 \bar{X}_4$
0	1	1	1	7	0	
1	0	0	0	8	0	
1	0	0	1	9	0	
1	0	1	0	M	0	
1	0	1	1	E	1	$X_1 \bar{X}_2 X_3 X_4$
1	1	0	0	L	1	$X_1 X_2 \bar{X}_3 \bar{X}_4$
1	1	0	1	O	0	
1	1	1	0	D	0	
1	1	1	1	OFF	1	$X_1 X_2 X_3 X_4$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 X_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 X_3 X_4$$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_3 X_4 (X_2 + \bar{X}_2)$$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_3 X_4$$

The logic behind the truth table for CYNTH stays the same as it was MELOD for everything except the 5 letters. Due to the limited number of segments, some letters may look different from what is expected. Most of them closely resemble their lowercase version except for C.

Table 7. Truth table for the Seven-Segment Decoder for segment 0 for CYNTH.

X1	X2	X3	X4	Output Display	Segment 0	Minterm
0	0	0	0	0	0	
0	0	0	1	1	1	$\bar{X}_1 \bar{X}_2 \bar{X}_3 X_4$
0	0	1	0	2	0	
0	0	1	1	3	0	
0	1	0	0	4	1	$\bar{X}_1 X_2 \bar{X}_3 \bar{X}_4$
0	1	0	1	5	0	
0	1	1	0	6	0	
0	1	1	1	7	0	
1	0	0	0	8	0	
1	0	0	1	9	0	
1	0	1	0	C	0	
1	0	1	1	Y	1	$X_1 \bar{X}_2 X_3 X_4$
1	1	0	0	N	1	$X_1 X_2 \bar{X}_3 \bar{X}_4$
1	1	0	1	T	1	$X_1 X_2 \bar{X}_3 X_4$
1	1	1	0	H	1	$X_1 X_2 X_3 \bar{X}_4$
1	1	1	1	OFF	1	$X_1 X_2 X_3 X_4$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 X_4 + X_1 X_2 \bar{X}_3 (\bar{X}_4 + X_4) + X_1 X_2 X_3 (\bar{X}_4 + X_4)$$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 X_4 + X_1 X_2 \bar{X}_3 + X_1 X_2 X_3$$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 X_4 + X_1 X_2 (\bar{X}_3 + X_3)$$

$$\text{Segment0} = \bar{X}_1 \bar{X}_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 X_4 + X_1 X_2$$

Table 8. Truth table for the Seven-Segment Decoder for segment 1 for CYNTH.

X1	X2	X3	X4	Output Display	Segment 1	Minterm
0	0	0	0	0	0	
0	0	0	1	1	0	
0	0	1	0	2	0	
0	0	1	1	3	0	
0	1	0	0	4	0	
0	1	0	1	5	1	$\bar{X}_1 X_2 \bar{X}_3 X_4$
0	1	1	0	6	1	$\bar{X}_1 X_2 X_3 \bar{X}_4$
0	1	1	1	7	0	
1	0	0	0	8	0	
1	0	0	1	9	0	
1	0	1	0	C	1	$X_1 \bar{X}_2 X_3 \bar{X}_4$
1	0	1	1	Y	0	
1	1	0	0	N	1	$X_1 X_2 \bar{X}_3 \bar{X}_4$
1	1	0	1	T	1	$X_1 X_2 \bar{X}_3 X_4$
1	1	1	0	H	0	
1	1	1	1	OFF	1	$X_1 X_2 X_3 X_4$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 X_4 + X_1 X_2 X_3 X_4$$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 X_4 (\bar{X}_3 + X_3)$$

$$\text{Segment1} = \bar{X}_1 X_2 \bar{X}_3 X_4 + \bar{X}_1 X_2 X_3 \bar{X}_4 + X_1 \bar{X}_2 X_3 \bar{X}_4 + X_1 X_2 \bar{X}_3 \bar{X}_4 + X_1 X_2 X_4$$

Part 7: Circuit in Verilog

In Verilog, when defining the inputs for x, the bits are reversed since the truth table uses X1 as the MSB (Most Significant Bit) and X4 as the LSB (Least Significant Bit). However, in Verilog, when defining x as 4-bits (x[3:0]), x[3] corresponds to X1 in the truth table (MSB) while x[0] corresponds to X4 in the truth table (LSB).

Code 1. Verilog code for Seven Segment Decoder for 0-9 and MELOD.

```
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
    reg [6:0] reg_LEDs;

    assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
        (x[2] & ~x[1] & ~x[0]) |
        (x[3] & x[2] & x[1]);
    assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
        (~x[3] & x[2] & x[1] & ~x[0]) |
        (x[3] & x[2] & ~x[1] & ~x[0]) |
        (x[3] & x[1] & x[0]);

    assign hex_LEDs[6:2]=reg_LEDs[6:2];

    always @(*)
    begin
        case (x)
            4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
            4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
            4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
            4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
            4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
            4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
            4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
            4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
            4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
            4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
            4'b1010: reg_LEDs[6:2]=5'b10010; //7'b1001000 letter M
            4'b1011: reg_LEDs[6:2]=5'b00001; //7'b0000110 letter E
```

```

    4'b1100: reg_LEDs[6:2]=5'b10001; //7'b1000111 letter L
    4'b1101: reg_LEDs[6:2]=5'b10000; //7'b1000000 letter O
    4'b1110: reg_LEDs[6:2]=5'b01000; //7'b0100001 letter D
    4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
    default: reg_LEDs[6:2] = 5'b11111;
endcase
end
endmodule

```

Code 2. Verilog code for Seven Segment Decoder for 0-9 and CYNTH.

```

module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
    reg [6:0] reg_LEDs;

    assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
        (~x[3] & x[2] & ~x[1] & ~x[0]) |
        (x[3] & ~x[2] & x[1] & x[0]) |
        (x[3] & x[2]);
    assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
        (~x[3] & x[2] & x[1] & ~x[0]) |
        (x[3] & ~x[2] & x[1] & ~x[0]) |
        (x[3] & x[2] & ~x[1] & ~x[0]) |
        (x[3] & x[2] & x[0]);

    assign hex_LEDs[6:2]=reg_LEDs[6:2];

    always @(*)
    begin
        case (x)
            4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
            4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
            4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
            4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
            4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
            4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5

```

```
4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
4'b0111: reg_LEDs[6:2]=5'b111110; //7'b11111000 decimal 7
4'b1000: reg_LEDs[6:2]=5'b00000; //7'b00000000 decimal 8
4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
4'b1010: reg_LEDs[6:2]=5'b10001; //7'b1000110 letter C
4'b1011: reg_LEDs[6:2]=5'b00100; //7'b0010001 letter Y
4'b1100: reg_LEDs[6:2]=5'b01010; //7'b0101011 letter N
4'b1101: reg_LEDs[6:2]=5'b00001; //7'b0000111 letter T
4'b1110: reg_LEDs[6:2]=5'b00010; //7'b0001001 letter H
4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
default: reg_LEDs[6:2] = 5'b11111;
endcase
end
endmodule
```

References

- [1] ChipVerify, "Verilog data types," ChipVerify,
<https://www.chipverify.com/verilog/verilog-data-types> (accessed Jan. 26, 2026).
- [2] A. Qadir, "Reg vs wire in Verilog HDL," Medium,
<https://medium.com/@aqibaabdulqadir/reg-vs-wire-in-verilog-hdl-resolved-dda3f6122e96> (accessed Jan. 26, 2026).
- [3] D. K. Tala, "Verilog HDL Syntax And Semantics Part-II," Verilog HDL syntax and semantics part-II, <https://www.asic-world.com/verilog/syntax2.html> (accessed Jan. 26, 2026).
- [4] ChipVerify, "Verilog blocking & non-blocking," ChipVerify,
<https://www.chipverify.com/verilog/verilog-blocking-non-blocking-statements> (accessed Jan. 26, 2026).
- [5] GeeksforGeeks, "Difference between combinational and Sequential Circuit," GeeksforGeeks,
<https://www.geeksforgeeks.org/digital-logic/difference-between-combinational-and-sequential-circuit/> (accessed Jan. 26, 2026).
- [6] VLSIFacts, "How to avoid latch inference in Verilog?," VLSIFacts,
<https://vlsifacts.com/how-to-avoid-latch-inference-in-verilog/> (accessed Jan. 26, 2026).
- [7] J. Rajewski, "Verilog operators," Alchitry,
<https://alchitry.com/tutorials/verilog/verilog-operators/> (accessed Jan. 26, 2026).
- [8] ChipVerify, "Verilog arrays and Memories," ChipVerify,
<https://www.chipverify.com/verilog/verilog-arrays-memories> (accessed Jan. 26, 2026).